Guía3. Informe final Proyecto APT Asignatura Capstone

Informe final Proyecto APT
El objetivo de este informe es que describas los aspectos más relevantes de tu Proyecto APT. Es importante que fundamenten las decisiones que tuviste que tomar a lo largo del proceso. A continuación, encontrarás distintos campos que deberás completar con la información solicitada, los que dan cuenta del resumen de tu proyecto APT y sus principales resultados.

Nombre del proyecto	ComplianceIA, una plataforma web de autodiagnóstico de madurez en gobernanza de inteligencia artificial para empresas chilenas, con validación de evidencias asistida por un modelo de lenguaje (LLM), anonimización local de datos personales, rutas formativas y un panel de estadísticas comparativas por rubro.
Área (s) de desempeño(s)	El proyecto se movió principalmente en el área de desarrollo de soluciones de software, porque construimos un backend completo con arquitectura por capas, base de datos, procesamiento asíncrono e integración con un proveedor de IA. También abordó la gestión de proyectos informáticos, ya que planificamos y controlamos el trabajo con Scrumban durante cuatro sprints, y la seguridad de la información, dado que el núcleo del producto es proteger datos personales bajo la Ley N.º 21.719. En menor medida tocamos el área de calidad de software, con una suite de pruebas automatizadas y mediciones de rendimiento como criterio de aceptación.
Competencias	Las competencias de nuestro plan de estudio que trabajamos en el proyecto fueron siete, y todos los integrantes participamos en cada una de ellas aunque con distinta intensidad según el rol.

	En **base de datos** remodelamos y construimos 19 tablas en PostgreSQL 16 para soportar la matriz de controles, las evaluaciones, las evidencias, las solicitudes ARCOP y la auditoría de cada llamada al modelo. Gestionamos el esquema con 13 migraciones versionadas en Alembic, cargamos los catálogos como datos semilla y optimizamos consultas críticas, como la que entrega los controles aplicables, que bajó de 542 ms a 24 ms. En **programación** desarrollamos el backend en Python con FastAPI y SQLAlchemy asíncrono, siguiendo Clean Architecture para separar la API, los casos de uso, el dominio y la infraestructura. Programamos 35 endpoints, un worker asíncrono con ARQ y Redis, y una suite de pruebas automatizadas con pytest que respalda cada historia de usuario. En **seguridad de sistemas informáticos** aplicamos privacidad desde el diseño: anonimización local antes de enviar o guardar cualquier texto, cifrado de la información sensible, un mapa de entidades irreversible, autenticación por roles, detección de inyección de instrucciones y un comportamiento *fail-closed*, en el que el sistema aborta antes que exponer un dato. También corregimos vulnerabilidades propias, como evitar que las claves quedaran dentro de la imagen de Docker. En **soluciones con inteligencia artificial** integramos un modelo de lenguaje de Groq para validar evidencias y un motor NER con spaCy para detectar datos personales en español. Diseñamos una capa de Políticas e IA con prompts versionados, validación estricta de las respuestas del modelo, métricas de cada llamada y revisión humana, bajo la regla de que el backend decide y el modelo solo propone. En **control de versiones con GitHub** trabajamos sobre el repositorio `jerReyDuoc/Capstone`, con la rama `main` como versión estable. Los cambios se revisaban y probaban localmente en Docker antes de integrarlos, y cada entrega importante quedó documentada en el README y en archivos de cambios por historia. En **gestión de proyectos** usamos Scrumban con roles definidos, épicas, historias con criterios de aceptación y puntos, un tablero con límite de trabajo en curso y una Definición de Hecho acordada por el equipo. Las revisiones periódicas del backlog nos permitieron medir el avance real y redistribuir la carga entre sprints. En **gestión de riesgos** identificamos y priorizamos los riesgos técnicos, de seguridad y de plazo en una tabla de deuda técnica, y tomamos decisiones para mitigarlos, como bajar la meta de TRL 6 a TRL 5 o adelantar historias de autorización para no sobrecargar el último sprint. Además, el propio producto evalúa la gestión de riesgos de las empresas, con un dominio completo basado en el NIST AI RMF.

Abstract	ComplianceIA is a web platform built for Therion Labs that allows Chilean organizations to self-assess their maturity in artificial intelligence governance. Companies are adopting AI tools without security policies, audits are still done by hand in spreadsheets, and the new Personal Data Protection Law (Law No. 21,719) now requires them to prove compliance. Our platform answers that need with an adaptive questionnaire of up to 40 controls drawn from Law 21,719, ISO/IEC 42001, the NIST AI Risk Management Framework and the OWASP Top 10 for LLM applications. Each organization uploads evidence that is validated asynchronously by a large language model, but only after a local named-entity recognition engine has replaced every piece of personal data with anonymous tokens. The backend decides the final result, while the model only proposes a classification. The team worked with Scrumban across four sprints and agreed on a technology readiness target of TRL 5, meaning components validated in a representative local environment. The resulting backend uses FastAPI, PostgreSQL, Redis and Docker, follows Clean Architecture, and is covered by an automated test suite of more than 300 cases. All twelve measured API endpoints respond with a 95th percentile under 70 ms against a 500 ms requirement, and the anonymization engine processes a page in about 25 ms with 0.988 precision and full personal data coverage. The main lesson for the team was that responsible AI is not a feature added at the end, but a set of design decisions taken from the first sprint. **Keywords:** AI governance, Law 21,719, data anonymization, NER, LLM, Scrumban, FastAPI.

Contenidos del informe final	
1. Relevancia del proyecto APT	Las organizaciones chilenas están incorporando inteligencia artificial sin la capacidad interna para gobernarla. Es común que se usen asistentes generativos con datos de clientes sin una política de uso, un responsable designado ni un registro de qué información sale de la empresa, y cuando se quiere medir la exposición, el diagnóstico se hace a mano en planillas, depende de quien lo aplica y es difícil de repetir. Como resume nuestro cliente, Therion Labs, la IA no falla por falta de tecnología, sino por falta de capacidad para gobernarla. El tema es relevante para la Ingeniería en Informática porque coincide con la plena vigencia de la Ley N.º 21.719 de protección de datos personales, prevista para diciembre de 2026, que crea una agencia fiscalizadora, endurece las sanciones y exige demostrar cumplimiento con evidencias. Para quienes desarrollamos software, la privacidad pasa a ser una condición del diseño, y saber integrar IA sin exponer datos personales es hoy un perfil muy demandado. La situación se ubica en Chile, con foco en la Región Metropolitana donde opera el cliente. El público principal son las pymes, que tienen equipos de TI reducidos, rara vez cuentan con un oficial de cumplimiento y no pueden pagar una auditoría

	externa cada vez que adoptan una herramienta. Los afectados directos son los encargados que responden el diagnóstico y los oficiales de cumplimiento; los indirectos, los clientes y trabajadores cuyos datos terminan procesados por sistemas de IA. El aporte de valor, en un contexto simulado pero con un cliente real, es convertir un diagnóstico manual y subjetivo en uno estandarizado, trazable y comparable. La empresa obtiene su puntaje de madurez por dominio, sus brechas, una ruta formativa y una comparación con su rubro sin exponer a otras organizaciones. Además, el producto cumple lo que exige: ninguna evidencia llega al modelo con datos personales en claro, todo queda auditado y los titulares pueden ejercer sus derechos ARCOP desde la plataforma.
2. Objetivos	**Objetivo general** Desarrollar una plataforma web de autodiagnóstico de madurez en gobernanza de inteligencia artificial para Therion Labs, que permita a las organizaciones chilenas identificar sus brechas de cumplimiento frente a la Ley N.º 21.719 y a estándares internacionales, validando sus evidencias con apoyo de un modelo de lenguaje sin exponer datos personales, y que alcance un nivel de madurez tecnológica TRL 5 en un entorno local representativo. Objetivos específicos Cada objetivo específico se asocia a una épica o a un sprint del proyecto: 1. Implementar un motor de autodiagnóstico con filtro de aplicabilidad y reglas de cálculo que entregue un puntaje de madurez por dominio, excluyendo los controles que no se aplican. 2. Integrar un modelo de lenguaje que valide evidencias de forma asíncrona, dejando la decisión final al backend y auditando cada llamada. 3. Implementar una capa de privacidad que anonimiza localmente los datos personales, bloquee la inyección de instrucciones y permita ejercer los derechos ARCOP de la Ley 21.719. 4. Generar reportes de madurez y brechas, rutas formativas y un benchmark por rubro que no revele datos de ninguna organización. 5. Asegurar la calidad con pruebas automatizadas, mediciones de rendimiento y un despliegue reproducible en contenedores. La meta inicial era TRL 6, pero el equipo la fijó en TRL 5 por el tiempo disponible, priorizando un producto bien validado en un entorno local por sobre un despliegue apurado sin las pruebas necesarias.
3. Metodología	Trabajamos con Scrumban. De Scrum tomamos los roles, los sprints con objetivo claro y la priorización del backlog por el Product Owner; de Kanban, el tablero

continuo (Por hacer, En curso, Listo) con límite de trabajo en curso, para terminar historias antes de abrir otras.

Jerson Reyes, como Product Owner, fue el puente con Therion Labs y tomó las decisiones de negocio, como el puntaje de un control sin evidencia o la irreversibilidad del mapa de entidades. Jason Sánchez, como Scrum Master, facilitó las reuniones, contrastó el backlog con el código y propuso ajustes de carga. Ángel Muñoz lideró el desarrollo técnico, aunque en la práctica los tres programamos, probamos y documentamos.

El trabajo se dividió en tres épicas (motor de autodiagnóstico, procesamiento inteligente con privacidad, y rutas formativas con panel de visualización), descompuestas en historias con criterios de aceptación y puntos, y estas en tareas técnicas. Los sprints quedaron así:

Sprint	**Objetivo**
Sprint 1	Desarrollo e implementación del backend y de las reglas de cálculo
Sprint 2	Despliegue del motor NER, gestión de derechos ARCOP y cifrado
Sprint 3	Integración de IA, controles, detección de brechas y benchmarking
Sprint 4	Despliegue del frontend, automatización del pipeline y entorno de staging

Acordamos como Definición de Hecho que una historia está terminada cuando sus criterios funcionan en el backend con pruebas automatizadas, dejando la interfaz para el cuarto sprint. Así medimos avance real y no pantallas que solo aparentan funcionar.

Scrumban era pertinente porque somos un equipo pequeño con disponibilidad variable por otras asignaturas: el flujo continuo nos dejó avanzar sin depender de ceremonias fijas, y los sprints nos dieron hitos claros para el docente y el cliente. Además, integrar IA implica mucha incertidumbre técnica, y una

	metodología ágil nos permitió ajustar el plan cada vez que una prueba mostraba un camino mejor.
4. Desarrollo	Etapas y actividades La primera etapa fue de levantamiento y diseño junto a Therion Labs. Construimos la matriz de 40 controles en cuatro dominios (Protección de Datos, Gobernanza de IA, Gestión de Riesgos y Uso Responsable), cada uno con su referencia normativa, evidencia esperada, criticidad, brecha y ruta formativa. También diseñamos el filtro de aplicabilidad, siete preguntas que dejan 22 controles fijos y activan 18 condicionales según el perfil de la empresa. En el primer sprint levantamos el backend en FastAPI con Clean Architecture, creamos las 19 tablas de PostgreSQL con migraciones Alembic y programamos las reglas de puntaje, que excluye los controles no aplicables. El segundo sprint se centró en la privacidad. Implementamos el motor NER con spaCy (`es_core_news_md`) y reglas para RUT, correos, teléfonos y direcciones, que reemplaza cada dato por etiquetas como `[RUT_ANON]` y aborta si falla. Sumamos el flujo de derechos ARCOP con código de seguimiento y plazos legales, e integramos nuestro módulo de detección de inyección de instrucciones. Luego construimos la capa de Políticas e IA, intermediaria con Groq, que versiona los prompts, valida las respuestas del modelo, registra métricas y permite la revisión humana. La validación corre en un worker asíncrono (ARQ y Redis), así la API responde de inmediato. Con esa base cerramos el cuestionario adaptativo, la matriz de cinco estados, la anonimización antes de persistir y la optimización del NER. El cuarto sprint, en curso, concentra la interfaz, la integración continua y el entorno de staging. Facilitadores Lo que más nos ayudó fue tener un cliente real y experto, miembro de la Comisión de Seguridad IA de la CCHIA, que nos ahorró semanas de interpretación normativa. En lo técnico, Docker Compose permitió que los tres levantáramos los seis servicios con un solo comando y la misma configuración; Clean Architecture nos dejó cambiar piezas sin tocar la lógica de negocio, y la suite de pruebas nos dio confianza para refactorizar. Dificultades En lo técnico, el modelo de spaCy no se instalaba por una incompatibilidad con Pydantic, y Groq deprecó el modelo que usábamos. El NER confundía "la Ley 21.719" con una dirección y cargos como "Presidenta del Directorio" con nombres. Además, el endpoint de controles aplicables tardaba 542 ms (sobre el límite de 500 ms) y el benchmark quedaba vacío porque ninguna evaluación se marcaba como completada.

La revisión del backlog mostró también riesgos de seguridad propios: las claves podían quedar dentro de la imagen de Docker y la mayoría de los endpoints aún no exigía autenticación. En lo organizacional, la mayor dificultad fue el tiempo compartido con otras asignaturas, que dejó al cuarto sprint con mucha carga.

Ajustes realizados

Instalamos spaCy desde su archivo publicado en GitHub y cambiamos el modelo a `openai/gpt-oss-safeguard-20b`, dejándolo configurable. En la tarea T-04 corregimos los falsos positivos del NER, acotamos su pipeline y lo precargamos al iniciar; descartamos ONNX porque el objetivo ya se cumplía con holgura.

Unificamos consultas y cachear el catálogo, bajando el endpoint lento de 542 ms a 24 ms. Agregamos un cierre definitivo de evaluaciones para alimentar el benchmark y corregimos las exclusiones de Docker. El Product Owner decidió además un mapa de entidades irreversibles, con solo huellas criptográficas y conteos, y los textos ARCOP cifrados como excepción.

En la gestión, bajamos la meta a TRL 5 y fijamos la Definición de Hecho en el backend. Como Scrum Master propuse adelantar al tercer sprint la autorización, el inicio público y el informe PDF para aliviar el cuarto, y sumamos ocho historias nuevas surgidas de la revisión, como integración continua y retención automática de datos.

5. Evidencias

El backend cumple los indicadores de calidad comprometidos en el diseño, y seis de las siete historias de los primeros sprints están verificadas contra el código:

Indicador	Requisito	Resultado obtenido	Evidencia
Latencia de la API REST (p95 por endpoint)	Menor a 500 ms	12 de 12 endpoints cumplen; el más lento, 52,1 ms	`reporte_latencia_live_hu01.md` (16-oct-2026, Docker, concurrencia 10)
Tiempo de anonimización por página	Menor a 2,0 s	25,1 ms (p95) por página de 3.000 caracteres	`CAMBIOS_T04.md`, `test_rendimiento_ner.py`

Precisión del motor NER	Sin sacrificar precisión	0,988 sobre el corpus de referencia	`tests/data/ner_gold.jsonl`, `benchmark_ner.py`
Cobertura de datos personales	Ningún dato en claro hacia el LLM	100 % de los datos del corpus anonimizados	`test_live_capa_ia.py` (13-oct-2026)
Suite automatizada	Toda historia respaldada por pruebas	336 pruebas aprobadas y 8 omitidas en la última corrida reportada	Salida de `pytest` en Docker
Madurez tecnológica	TRL 5	Componentes validados con PostgreSQL, Redis, spaCy y Groq reales en entorno local	Prueba en vivo de extremo a extremo

La evidencia más completa es la prueba en vivo del 13 de octubre de 2026: un acta con datos personales ficticios se sube por la API, se anonimiza, la válida Groq y el resultado vuelve a la base de datos. Comprobó que el modelo nunca devolvió datos personales, que no quedó ningún valor en claro en la base y que un intento de inyección fue bloqueado antes de llamar al modelo.

Están completas en el backend las historias HU-01 (cuestionario adaptativo), HU-02 (matriz de cinco estados), HU-03 (puntaje sin N/A), HU-05 (NER local), HU-06 (anonimización antes de persistir) y HU-08 (derechos ARCOP); el aviso legal (HU-04) pasó al siguiente sprint. El frontend actual es un prototipo con datos simulados, y su integración es el objetivo del cuarto sprint.

Como anexo se desarrolló al final del informe con las diferentes capturas de los siguientes puntos::

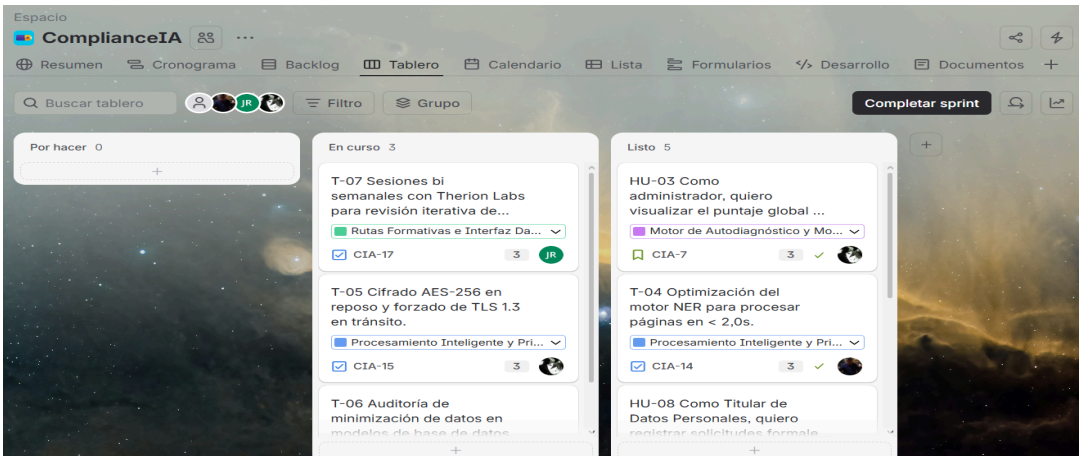
- Figura 1. Tablero Scrumban con las columnas Por hacer, En curso y Listo, y el backlog con historias, puntos y criterios de aceptación.
- Figura 2. Los seis servicios corriendo con `docker compose ps`.
- Figura 3. Documentación interactiva de la API en Swagger (`/docs`), con los 35 endpoints.

	• Figura 4. Resultado de la suite completa con `docker compose exec worker pytest tests -v -rxX`. • Figura 5. Reporte de latencia con los 12 endpoints bajo 500 ms. • Figura 6. Historial de commits del repositorio `jerReyDuoc/Capstone` y el README técnico.
6. Intereses y proyecciones profesionales	El proyecto nos mostró que el cruce entre inteligencia artificial y regulación es un campo profesional real y en crecimiento. Empezamos con intereses generales en desarrollo de software y terminamos con ámbitos más específicos que se complementan entre sí. **Jason Sánchez (Scrum Master).** Llegué interesado en el desarrollo backend, pensando que la gestión ocurría alrededor del código. Contrastar cada historia con el código, detectar riesgos como el benchmark vacío o la sobrecarga del cuarto sprint y proponer cómo redistribuirla me mostró que gestionar un proyecto técnico exige entenderlo técnicamente. Mis intereses se ampliaron hacia el aseguramiento de calidad, la seguridad en sistemas con IA y la gestión ágil, y me proyecto como líder técnico o analista de QA y seguridad, con una certificación ágil y estudios en ciberseguridad. **Jerson Reyes (Product Owner).** Traducir las necesidades de un cliente experto en decisiones de producto me acercó al análisis de negocio, la protección de datos y el cumplimiento normativo. Me proyecto en roles de producto o consultoría en gobernanza de datos e IA, profundizando en la Ley 21.719 e ISO/IEC 42001. **Ángel Muñoz (Developer Lead).** Liderar el desarrollo confirmó mi interés por la arquitectura de software y la integración de modelos de lenguaje en sistemas reales que se puedan mantener y auditar. Me proyecto como desarrollador backend o ingeniero de IA aplicada, profundizando en arquitecturas en la nube, MLOps e integración de LLM con foco en privacidad.
7. Individual Conclusions	**Jason Sánchez, Scrum Master** Working as Scrum Master taught me that a project is only as healthy as the honesty of its backlog. When I reviewed every user story against the code, I found gaps behind stories marked as done, such as an empty benchmark or personal data stored in plain text, and raising them early let us fix them before the final delivery. I learned that facilitating means proposing options, like rebalancing sprints or lowering the target to TRL 5, and letting the Product Owner decide. Contributing to the QA and latency tests also showed me that quality only matters when it can be measured. I finish more confident leading technical teams and interested in quality assurance and AI security. **Jerson Reyes, Product Owner** As Product Owner I was the voice of Therion Labs inside the team, which made me judge every feature by the value it brings to a real organization. Many key decisions were legal and business ones, like whether the entity map should be

	reversible, and I learned to make them with incomplete information and document them. The project confirmed my interest in data governance and product management. **Ángel Muñoz, Developer Lead** Leading development was the most demanding technical challenge of my degree: an API, a background worker, a database, a queue, a local language model and an external AI provider had to work together reliably in Docker. Clean Architecture proved its value whenever we replaced a component, and optimizing the NER engine showed me that performance and privacy can coexist, at about 25 ms per page with 0.988 precision. I finish with stronger backend skills and a clear interest in applied AI engineering.
8. Reflection	The most valuable lesson of this project was that responsible AI is a design decision, not an extra feature. We started by seeing the language model as the center of the product and ended up understanding that most of the work lies around it: deciding what it can see, validating what it returns, recording every call and letting a human override it. The rule that the backend decides and the model only proposes is something we will carry into future projects. We also learned to be realistic about scope. Lowering the target to TRL 5 and defining done as working backend with automated tests let us deliver a product that is actually tested, instead of screens that hide unfinished logic. An agile methodology made these adjustments possible every time a test or review showed a better path. Finally, technical skills were not enough on their own: we had to negotiate priorities, exchange critical feedback and share roles, since in a team of three everyone does a bit of everything. Working with a real client, and knowing that Law 21,719 will soon affect thousands of Chilean organizations, gave our work purpose. We leave better prepared as professionals who understand the ethical and legal impact of what we build.

Anexos Evidencias

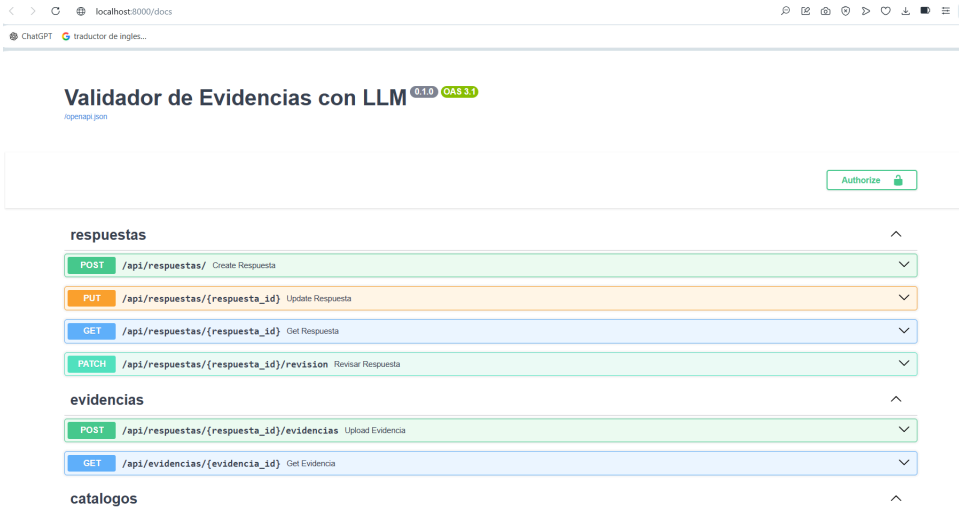
- Figura 1. Tablero Scrumban



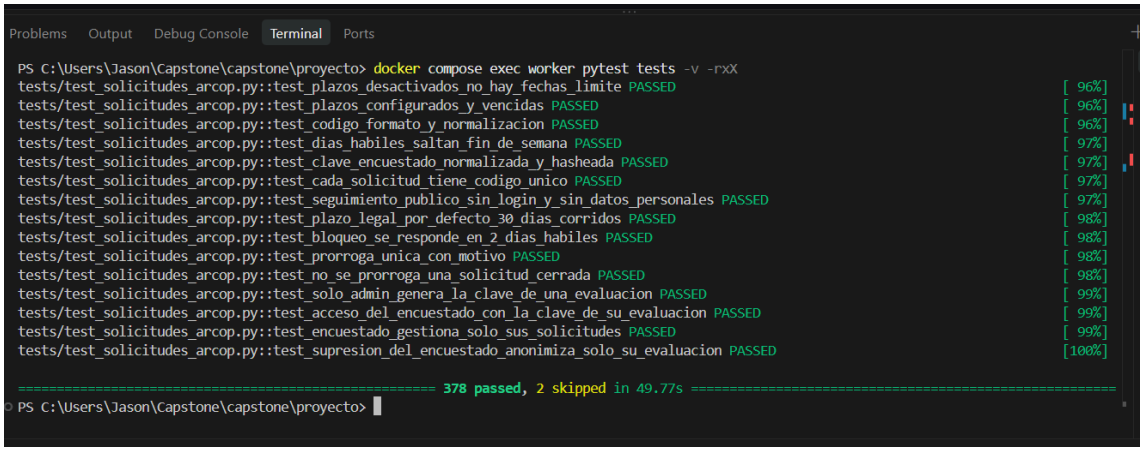
- Figura 2. Los seis servicios corriendo con `docker`

```
PS C:\Users\Jason\Capstone\capstone\proyecto> docker compose up -d --build
 => => exporting layers
 => => exporting manifest sha256:ef68dc97faea978e0fe25badb694026a14a2202603e128d05ab8be17a22473fa
 => => exporting config sha256:6b5128e74a02fbbe49a9e03e89c83e0b04941f5dcc17118265a5307b4f1ee709
 => => exporting attestation manifest sha256:9f47a221fb90a14dffb446479e631e0dc1129c906fed69faa6cb9a1de050a7c9
 => => exporting manifest list sha256:41f52a0084505aeeb1a272cf7ff20cf63290d56443aa77259e0c4126156701da
 => => naming to docker.io/library/complianceai-backend:latest
 => => unpacking to docker.io/library/complianceai-backend:latest
 => [migrate] resolving provenance for metadata file
 => [worker] resolving provenance for metadata file
 => [api] resolving provenance for metadata file
[+] up 6/7
 ✓ Image complianceai-backend:latest Built
 ✓ Container proyecto-db-1           Healthy
 ✓ Container proyecto-redis-1        Healthy
 ✓ Container proyecto-migrate-1      Exited
 ✓ Container proyecto-api-1          Healthy
 ✓ Container proyecto-worker-1       Started
 - Container proyecto-frontend-1     Starting
```

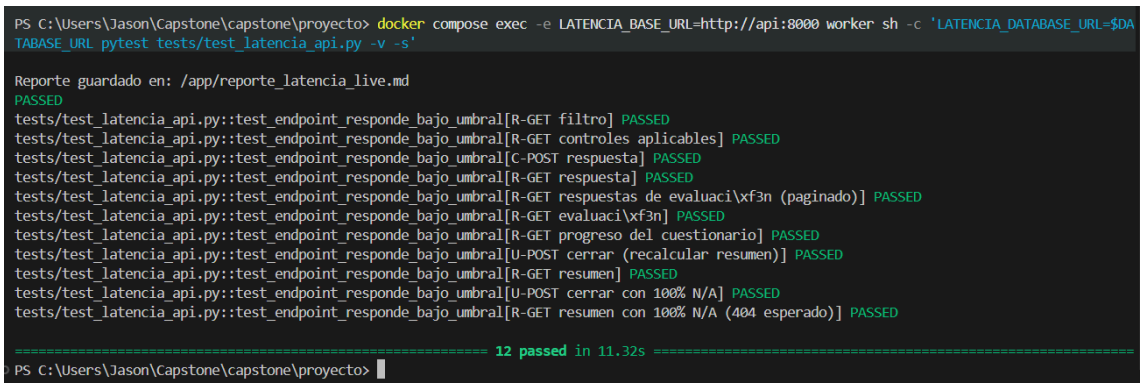
- Figura 3. Documentación interactiva de la API en Swagger



- Figura 4. Resultado de la suite completa con `docker`



- Figura 5. Reporte de latencia con los 12 endpoints



- Figura 6. Historial de commits del repositorio y el README

Historial Commits



README del proyecto

